

ai/ ハーネス

Claude と Codex が 1 つのファイルベースの手順を共有するしくみ – 各部分の役割、依頼がたどる流れ、そしてこの形にした理由。

ai/project.md

ai/policies.md

ai/rules/

ai/workflows/

ai/skills/

ai/templates/

ai/checklists/

ai/evaluations/

ai/improvements/

2 つのエージェント、多数のセッション、共有されない記憶

このリポジトリでは Claude と Codex の両方が、多くの場合別々のセッションで作業する。ファイルに基づく共通の指示がなければ、各セッションは重要な事柄を導き直すか、気づかないまま食い違ってしまう。

- **範囲** – この作業に含まれるものと含まれないもの
- **技術選定** – プロジェクトが実際に決めたことと、まだ未決のこと
- **品質基準** – 変更の種類ごとの「正しい」と「完了」の意味
- **権限** – エージェントが単独でできること、止まって確認すべきとき

ハーネスの役割

依頼とリポジトリ自身のファイルがあれば、どのエージェントでも、どのセッションでも、過去のチャット履歴を読まずに正しく一貫した作業を引き継げること。

文脈を永続的（ファイルに）かつエージェント非依存（両者が読む 1 つの原本）にする。

ai/ にあるもの - 各部分が存在する理由

ai/project.md

確定した技術判断と未決の技術判断

プロジェクトが決めていない選定を、エージェントが作ったり仮定したりしない

ai/policies.md

権限、計画の承認、一時停止の条件、信頼できない内容への対策

安全の境界：単独で行うか、止まって確認するか

ai/rules/ ・ 8 ファイル

common、documentation、frontend、backend、database、testing、git-review、ci-cd

領域ごとに 1 つの現行基準を、すべてのスキルが適用する

ai/workflows/ ・ 4

bootstrap、feature-delivery、bug-fix、harness-improvement

作業の始まり方ごとの出発点となる手順

ai/skills/ ・ 13

「X をどう行うか」の手順

範囲が決まり確認できる単位。8 部構成で共通

ai/templates/

各スキルの成果物の出発点となる文書

実在する形式に倣う（MADR、IEEE 829、基本設計書 …）

ai/checklists/ ・ 4

その時点での合否の関門

「ルールに従ったか」を明示的な確認にする

ai/evaluations/

手動のシナリオ：手順のルールと攻撃的なケース

指針の変更は信頼ではなくテストで確かめる

ai/improvements/

ハーネス変更ごとに 1 つの記録

ハーネスがなぜ変わったかの履歴を残す

3 つの異なる問い、3 つの異なる場所

1 つにまとめると、権限が技術的な詳細に埋もれるか、すべてのルールが確認の機会のない提案になってしまう。

ai/policies.md

「このエージェントはこれを行ってよいか、それとも止まって確認すべきか？」

計画の承認、一時停止の条件、外部操作（push、PR、マージ、デプロイ）、信頼できない内容はデータであり指示ではない。

ai/rules/

「この種の変更で正しい形はどうあるべきか？」

RFC 9457 のエラー、厳格な TypeScript、パラメータ化 SQL、WCAG 2.2 AA、SHA 固定のアクション、テストピラミッド …

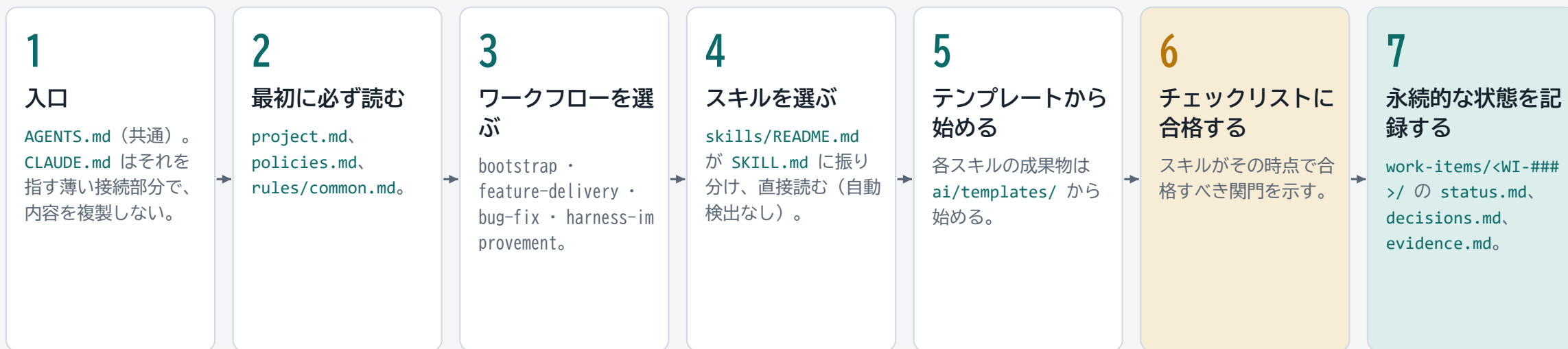
ai/checklists/

「今、次のステップの前に、その基準を実際に満たしたか？」

design-consistency、security-review、delivery、release-readiness：必要な瞬間の明示的な関門。

依頼が実際にたどる流れ – どちらのエージェントでも

すべての作業は同じ流れで始まる。順序に意味があり、各ステップが次に読むものを絞り込む。



作業ごとの流れの外： ai/evaluations/ はハーネスが自分自身を確認するしくみで、共有の指針の変更が提案されたときに実行する。ステップ 7 が作業を再開可能にする：次のステップは、別のセッションの別のエージェントかもしれないが、チャットではなくリポジトリから続ける。

作業の 4 つの始まり方、4 つの出発点の手順

各ワークフローは作業に必要なスキルを順に並べ、明示的な終了条件で終わる – すべての道筋を固定はしない。

project-bootstrap

- 1 未決の技術判断とデモの範囲を確認
- 2 計画：骨組み、ローカル環境、CI の確認
- 3 範囲の決まった承認済みのステップで構築
- 4 実際のコマンドを検証し project.m d を更新
- 5 不足を記録し feature-delivery に引き継ぐ

feature-delivery

- 1 要件と受け入れ基準
- 2 計画を示し、承認を得る
- 3 アーキテクチャ確認 → BD / DB / API / DD
- 4 設計を突き合わせてから実装
- 5 単体・結合、次にシステム・E2E
- 6 差分をレビューし、権限内で PR
- 7 証跡を記録

bug-fix

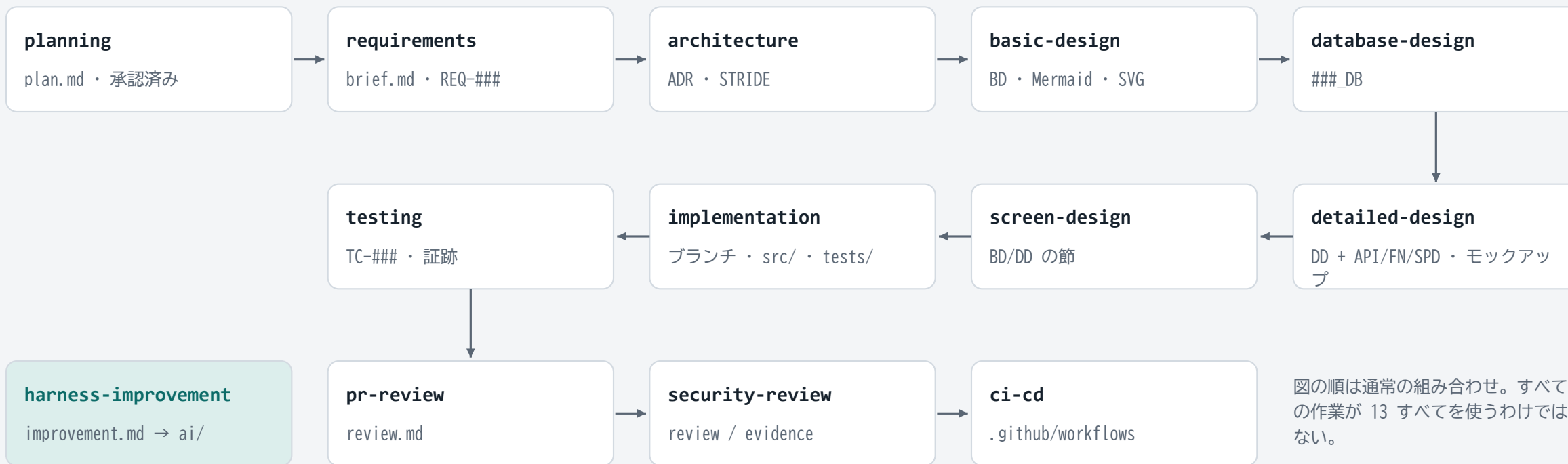
- 1 失敗と期待の動作を再現できるケース
- 2 重大度：稼働中の障害はまず緩和
- 3 原因を見つけ、回帰の確認を追加
- 4 修正し、影響する設計・契約を更新
- 5 対象の確認を実行し、証跡を記録
- 6 稼働中なら責めない事後分析

harness-improvement

- 1 観察した失敗・変化を記録
- 2 最小の有益な変更を提案
- 3 ai/evaluations のケースで評価
- 4 限界を記録し、採用前にレビュー
- 5 バージョン管理でロールバック – 関門は緩めない

13 のスキルを、再開できる引き継ぎの連鎖として組み合わせる

ワークフローが作業に必要なものを選ぶ。各スキルのステップ 8 が成果物を次に渡すため、エージェントやセッションが替わっても連鎖は途切れない。



独自のきっかけで動く：ハーネス自体が誤っているか古いとき。

すべての SKILL.md は、意図して同じ 8 部構成を持つ

1-2

目的・入力と参照元の順序

始める前に必要なものを限定する。

3

実行の手順と適用するルール

各ステップをそれを定めるルールに結び付け、暗黙の「現行基準」を残さない。

4

ツールと環境

欠けているツールがあると何ができないかを正直に示す。

5

成果物、テンプレート、ID、保存場所

REQ BD DD DB SCR TC DEC WI の ID と場所を推測に任せず明示する。

6-7

チェックリスト・終了と失敗時

「完了」を主張ではなく確認できるものにする。

8

作業項目の更新と引き継ぎ

エージェントやセッションをまたいで流れを再開できるようにする。

8 つのルールファイル：領域ごとに 1 つの現行基準

スキルは基準を書き直さずにこれを参照するため、同じ変更は常に同じ基準を満たす。

common.md

承認された範囲を守る ・ 安定した ID、複製せずリンク ・ 未解決の判断を記録 ・ 焦点を絞った変更

documentation.md

英語が原本 ・ 全文書の英日 PDF ・ 日本語 UI の引用に英語の対訳 ・ 番号を先頭に
した ID、番号ごとに 1 フォルダ

frontend.md

Vite + TypeScript、厳格 (any なし) ・ WCAG 2.2 AA ・ 設定は import.meta.env、
ブラウザに秘密情報を置かない

backend.md

.NET 10 ・ nullable 型 ・ RFC 9457 のエラー、詳細を漏らさない ・ 構造化ログ ・
新しいエンドポイントに OpenTelemetry

database.md

PostgreSQL ・ パラメーター化 SQL のみ ・ 使用中のテーブルは expand/contract ・
最小権限のアプリロール ・ 先に復旧の限界

testing.md

受け入れ基準に結び付ける ・ ピラミッド型 ・ リセットできるデータ ・ 不安定なテ
ストは隔離 ・ 未実行 ≠ 合格

git-review.md

作業項目ごとにブランチと worktree ・ 小さく焦点を絞った PR ・ 直接 push しない
・ squash マージ ・ レビュー ≠ マージ

ci-cd.md

SHA 固定のアクション ・ 既定の権限は読み取り専用 ・ 長期の秘密情報より OIDC ・
非 root のスキャン済みイメージ

4 つの関門、それぞれ必要な瞬間に

その時点での合否の確認 - 「ルールに従ったか」を記憶に頼らない。

design-consistency

依存する実装の前

- 安定した ID と受け入れ基準
- DD が BD と一致、API と DB が一致
- 判断を解決済み、テストが設計に対応
- セキュリティ・個人情報、WCAG、マイグレーション、トレースを規定

security-review

セキュリティに関わる変更のマージ前

- 変更したすべてのエンドポイントに認証・認可
- 入力を検証、連結したクエリなし
- 秘密情報なし、審査済みの依存関係
- 情報を漏らすエラーなし、最小権限、脅威モデル

delivery

完了を報告する前

- 範囲と計画のリビジョンが特定できる
- 確認を記録、未実行には理由
- 権限の範囲内にとどまった
- 別のエージェントがファイルから続けられる

release-readiness

承認されたデプロイの前

- 対象の設定とマイグレーション計画を確認
- ロールバックが記述だけでなく実行できる
- スモークテストの基準と監視を定義
- イメージ・バージョンを PR までたどれる

すべての成果物は、実在する名前のある文書形式から始まる

成果物の目的に合わせて選び、形だけでなく役立つものにする。

テンプレート	倣う形式	記録するもの
<code>brief.md</code>	PRD の慣習	成功・失敗の基準付きの REQ-###、「行わないこと」
<code>plan.md</code>	PMI 式の実装計画	検証付きのステップ、承認された操作、リビジョンを保持
<code>status.md</code>	RAG 状況報告	状態、成果、リスク、1 つの次の行動
<code>decisions.md</code>	意思決定記録	背景、選択肢、答え、理由、影響
<code>evidence.md</code>	トレーサビリティマトリクス + テスト記録	REQ → 設計 → コード → テスト、実際の結果
<code>architecture-decision.md</code>	MADR	要因、選択肢、結果、帰結
<code>basic-design.md</code>	基本設計書	業務の流れ、画面、Mermaid の図、SVG ワイヤフレーム
<code>detailed-design.md</code> + DD/*	詳細設計書 一式	メインの DD + API ・ 機能 ・ 画面処理
<code>database-design.md</code>	テーブル定義書 + ER	テーブル、インデックス、制約、マイグレーションの限界
<code>test-plan.md</code>	IEEE 829-1998	対象、方針、ケース、結果、不足
<code>review.md</code>	Google のコードレビューの観点	設計、機能、複雑さ、テスト …
<code>improvement.md</code>	Rust 式の RFC	動機、代替案、評価、採用

記憶はチャットではなく作業項目にある

作業の途中で Claude から Codex に替わっても、やり直さずに作業項目を読んで続ける。

`work-items/<WI-###>/`

brief.md

何を、なぜ：REQ-### と受け入れ基準

plan.md

各リビジョンは作業前に示して承認を得る。すべて古い順に保持

status.md

現在の状態、リスク、1 つの次の行動

decisions.md

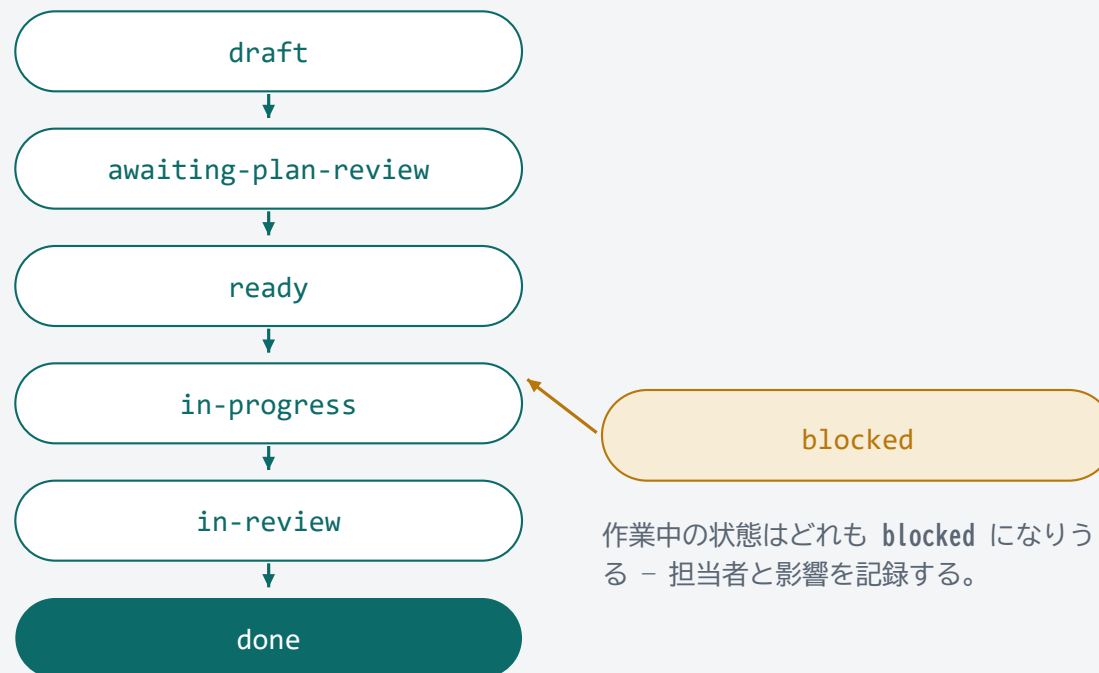
エージェントを止めたすべての問いとその答え

evidence.md

「完了」を後から確認できるようにするもの

+ `docs/en/...` の設計文書（BD、DB、DD、ADR、テスト）。それぞれ英日の PDF 付き

`status.md` の状態



作業中の状態はどれも **blocked** になりうる - 担当者と影響を記録する。

すべてのファイルの背後にある 5 つの原則

1

エージェントに依存しない、共有の 1 つの原本

手順は ai/ だけに置く。接続部分（AGENTS.md、CLAUDE.md、.claude/、必要になれば .codex/）はそこに振り分けるだけで、複製しない - エージェント固有のツールは共有のスキルに入れない。

2

Markdown は契約であり、実行者ではない

ハーネスは正しさと完了の意味を定める。作業はスクリプト、テストランナー、GitHub Actions が行い、その実際の出力を仮定ではなく記録する。

3

承認された計画が権限の範囲

ステップごとの再承認は不要だが、計画の範囲と一時停止の条件は厳格な限界。エージェントが単に読んだ内容は権限にならない。

4

結論には証拠が必要

「完了」「合格」「デプロイ済み」は、記録された実際に実行した確認の後にだけ主張する。未実行は未実行として記録する。

5

ハーネスは記録に残る形で自らを改善する

不足は harness-improvement スキルと評価ケースで直す - 共有の指針を黙って書き換えない。

一連の流れを 4 回実行 – Screen A、B、C は完了

WI-001 ~ WI-004 ・ それぞれの実行で確かめたこと

- WI-001 – project-bootstrap：骨組み、認証の基盤、ローカルの Compose 環境、CI
- WI-002 ・ Screen A – 最初のエンドツーエンドの機能：要件 → 計画 → BD → DB → 4 つの DD 文書 + モックアップ → コード → axe を含む単体・結合・E2E テスト → チェックリスト → PR
- WI-003 ・ Screen B – 既存の画面の上に作る：稼働中のスキーマの変更、共通のモジュールとエンドポイントの再利用、Screen A の設計の修正
- WI-004 ・ Screen C – 1 つの一貫したスナップショットに基づく読み取り専用のダッシュボード、モックアップレビュー後の設計の修正、全画面のナビゲーションバー

各実行は `work-items/WI-00N/` に記録されている。

現在の状況

CI は合格 – master へのすべてのプルリクエストでバックエンド、フロントエンド、E2E (Compose + Playwright) を実行。文書だけの変更では実行しない。

不足は記録に残る形で修正 – 流れの中で見つかった不足は、共有の指針を黙って書き換えず、harness-improvement で直した。

プロジェクトの未決事項 – デプロイ先、マージ・デプロイ権限、デモ動画、完全なロール設計 – 決めるのはオーナー。

次

固定したロードマップは完了し、次の作業項目は計画されていない。ハーネスで未検証なのはデプロイで、`release-readiness` と `ci-cd` スキルのデプロイの部分はデプロイ先の決定を待っている。